

# Let's test our understanding

- Can two threads run `b()` and `c()` concurrently on the same `Example` instance?

```
public class Example {  
    private int x = 2;  
    private int y = 3;  
  
    public synchronized void b() {  
        x = x + y;  
    }  
  
    public synchronized void c() {  
        y = 2 * x;  
    }  
    ...  
}
```

## Let's test our understanding

- Can two threads run b() and c() concurrently?

Not on the same instance, by def. of synchronized

```
public class Example {  
    private int x = 2;  
    private int y = 3;  
  
    public synchronized void b() {  
        x = x + y;  
    }  
  
    public synchronized void c() {  
        y = 2 * x;  
    }  
    ...  
}
```

## Let's test our understanding

- What are the possible values of  $[x,y]$  if  $b()$  and  $c()$  are run on the same `Example` instance?

```
public class Example {  
    private int x = 2;  
    private int y = 3;  
  
    public synchronized void b() {  
        x = x + y;  
    }  
  
    public synchronized void c() {  
        y = 2 * x;  
    }  
    ...  
}
```

# Let's test our understanding

- What are the possible values of [x,y] if b() and c() are run on the same Example instance?

```
public class Example {
    private int x = 2;
    private int y = 3;

    public synchronized void b() {
        x = x + y;
    }

    public synchronized void c() {
        y = 2 * x;
    }

    ...
}
```

Only two possible interleavings

Interleaving #1:  
First b() → [5,3]  
Then c() → [5,10]

Interleaving #2:  
First c() → [2,4]  
Then b() → [6,4]

# Let's test our understanding

- Can two threads run a() and b() concurrently on the same Example instance?

```
public class Example {  
    private int x = 2;  
    private int y = 3;  
  
    public void a() {  
        y = 2 * x;  
    }  
  
    public synchronized void b() {  
        x = x + y;  
    }  
    ...  
}
```

# Let's test our understanding

- Can two threads run a() and b() concurrently?

Yes, even on same instance: a() is not synchronized

```
public class Example {  
    private int x = 2;  
    private int y = 3;  
  
    public void a() {  
        y = 2 * x;  
    }  
  
    public synchronized void b() {  
        x = x + y;  
    }  
    ...  
}
```

## Let's test our understanding

- What are the possible values of  $[x,y]$  if  $a()$  and  $b()$  are run concurrently on the same instance?

```
public class Example {  
    private int x = 2;  
    private int y = 3;  
  
    public void a() {  
        y = 2 * x;  
    }  
  
    public synchronized void b() {  
        x = x + y;  
    }  
    ...  
}
```

# Let's test our understanding

- What are the possible values of  $[x,y]$  if  $a()$  and  $b()$  are run concurrently on the same instance?

```
public class Example {
    private int x = 2;
    private int y = 3;

    public void a() {
        y = 2 * x;
    }

    public synchronized void b() {
        x = x + y;
    }
    ...
}
```

Effectively, no synchronization → all possible interleavings of atomic actions

Outcome:  $[5,4]$  is also possible

E.g., interleaving #3:  
 $a()$  reads  $x = 2$   
 $b()$  sets  $x = 5$   
 $a()$  sets  $y = 2 * 2 = 4$



# Let's test our understanding

- Is the Example class thread safe?

```
public class Example {  
    private int x = 2;  
    private int y = 3;  
  
    public void a() {  
        y = 2 * x;  
    }  
  
    public synchronized void b() {  
        x = x + y;  
    }  
    ...  
}
```

# Let's test our understanding

- Is the Example class thread safe?

```
public class Example {
    private int x = 2;
    private int y = 3;

    public void a() {
        y = 2 * x;
    }

    public synchronized void b() {
        x = x + y;
    }
    ...
}
```

Technically, depends on the specifications.

Note however that threads can interfere when running a(), and generate values of [x,y] impossible otherwise: so, in practice, we can say that the class is not thread safe.